Below is a **master table of 300+ Java + Spring + Spring Boot + Microservices (Spring Cloud) + Kafka annotations** with **real-time use cases** (primary + secondary / supporting annotations).

✅ I'm including:

- **Core Java (SE)**
- **Jakarta / Java EE (JPA, Validation, REST, CDI, Transactions, etc.)**
- **Spring Core / DI**
- **Spring Boot**
- **Spring Web / MVC / REST**
- **Spring Data (JPA, Mongo, Redis, Elasticsearch)**
- **Spring Security / OAuth2**
- **Spring AOP**
- **Scheduling / Async / Caching**
- **Testing**
- **Microservices / Spring Cloud (Eureka, Config, Gateway, Circuit Breaker, Feign, Sleuth/Micrometer)**
- **Kafka (Spring Kafka)**
- **Swagger/OpenAPI**
- **Lombok** (widely used in real projects)

✅ 300+ Java + Spring + Spring Boot + Microservices + Kafka Annotations (Master Table)

Legend:

- **Primary** = commonly used annotation
- **Secondary** = supporting / advanced annotation

1) Core Java (java.lang / standard usage)

Annotation	Category	Real-time Use Case
@Override	Java	Ensures method actually overrides parent method (prevents signature mistakes)
@Deprecated	Java	Mark old APIs not to use (warn consumers)
@SuppressWarnings	Java	Suppress compiler warnings (unchecked, rawtypes, etc.)
@FunctionalInterface	Java	Ensure interface has exactly one abstract method (lambda support)

@SafeVarargs	Java	Avoid heap pollution warnings in varargs generics methods
@Native	Java	Marks constant used by native code (JNI)
@Repeatable	Java	Allows repeating same annotation multiple times
@Target	Java Meta	Restrict where annotation can be applied
@Retention	Java Meta	Control lifecycle of annotation (SOURCE/CLASS/RUNTIME)
@Inherited	Java Meta	Makes annotation inheritable to subclasses
@Documented	Java Meta	Include annotation in Javadoc
@ElementType	Java Meta	Used with @Target
@RetentionPolicy	Java Meta	Used with @Retention

---

2) Lombok (very common in Spring Boot projects)

Annotation	**Category**	**Real-time Use Case**
@Getter	Lombok	Auto-generate getters
@Setter	Lombok	Auto-generate setters
@ToString	Lombok	Auto-generate toString()
@EqualsAndHashCode	Lombok	Auto-generate equality methods
@NoArgsConstructor	Lombok	Generate default constructor
@AllArgsConstructor	Lombok	Generate constructor with all fields
@RequiredArgsConstructor	Lombok	Generate constructor with final/@NonNull fields (best for DI)
@Data	Lombok	@Getter @Setter @ToString @EqualsAndHashCode etc.

@Builder	Lombok	Builder pattern (DTO creation clean)
@SuperBuilder	Lombok	Builder for inheritance classes
@Slf4j	Lombok Logging	Adds log variable for SLF4J logging
@Log4j2	Lombok Logging	Adds log variable for Log4j2
@CommonsLog	Lombok Logging	Adds Apache commons logger
@NonNull	Lombok	Null checks on params/fields
@Value	Lombok	Immutable class (final fields, no setters)
@With	Lombok	Create "withField()" copy method
@SneakyThrows	Lombok	Throw checked exceptions without declaring
@Synchronized	Lombok	Safe synchronization wrapper
@Cleanup	Lombok	Auto-close resources
@FieldDefaults	Lombok	Set default field visibility
@Accessors(chain=true)	Lombok	Fluent setters (setX().setY())

---

3) Jakarta Persistence / JPA (Database + ORM layer)

Annotation	**Category**	**Real-time Use Case**
@Entity	JPA	Marks ORM entity class
@Table	JPA	Custom table name/indexes
@Id	JPA	Primary key
@GeneratedValue	JPA	Auto-generate PK
@Column	JPA	Column properties (nullable, length, etc.)
@Transient	JPA	Non-persistent field

@Embedded	JPA	Embedded object
@Embeddable	JPA	Define reusable embedded type
@Version	JPA	Optimistic locking
@OneToOne	JPA	1-1 relationship
@OneToMany	JPA	1-N relationship
@ManyToOne	JPA	N-1 relationship
@ManyToMany	JPA	N-N relationship
@JoinColumn	JPA	FK mapping
@JoinTable	JPA	N-N join table mapping
@Enumerated	JPA	Store enum
@Lob	JPA	Store large object (TEXT/BLOB)
@Temporal	JPA	Date/time mapping for legacy Date
@MappedSuperclass	JPA	Base entity mapping
@Inheritance	JPA	Inheritance strategy
@DiscriminatorColumn	JPA	For single-table inheritance
@DiscriminatorValue	JPA	Discriminator value
@SecondaryTable	JPA	Map entity to multiple tables
@NamedQuery	JPA	Predefined JPQL query
@NamedNativeQuery	JPA	Predefined SQL query
@SqlResultSetMapping	JPA	Mapping native query results
@EntityListeners	JPA	Hooks for auditing
@PrePersist	JPA	Before insert callback
@PostPersist	JPA	After insert callback
@PreUpdate	JPA	Before update callback

@PostUpdate	JPA	After update callback
@PreRemove	JPA	Before delete callback
@PostRemove	JPA	After delete callback
@PostLoad	JPA	After entity load callback

---

4) Jakarta Validation (Bean validation)

Annotation	**Category**	**Real-time Use Case**
@Valid	Validation	Validate nested objects in request
@NotNull	Validation	Field cannot be null
@NotBlank	Validation	String cannot be null/empty/whitespace
@NotEmpty	Validation	Collection/string must not be empty
@Size	Validation	Min/max length
@Min	Validation	Minimum value
@Max	Validation	Maximum value
@Positive	Validation	Must be positive
@PositiveOrZero	Validation	Must be >= 0
@Negative	Validation	Must be negative
@Email	Validation	Validate email pattern
@Pattern	Validation	Regex validation
@Past	Validation	Date must be in past
@PastOrPresent	Validation	Date <= now
@Future	Validation	Date must be future
@FutureOrPresent	Validation	Date >= now
@AssertTrue	Validation	Must be true

@AssertFalse	Validation	Must be false
@Digits	Validation	Digits constraints
@DecimalMin	Validation	BigDecimal min
@DecimalMax	Validation	BigDecimal max

---

5) Spring Core / DI / Beans

Annotation	**Category**	**Real-time Use Case**
@Component	Spring Core	Generic Spring Bean
@Service	Spring Core	Service-layer bean
@Repository	Spring Core	DAO bean + exception translation
@Controller	Spring MVC	MVC controller
@RestController	Spring MVC	REST controller (@Controller + @ResponseBody)
@Configuration	Spring Core	Java config class
@Bean	Spring Core	Declare bean manually
@Autowired	Spring DI	Inject dependencies
@Qualifier	Spring DI	Choose specific bean when multiple exist
@Primary	Spring DI	Default bean when multiple candidates
@Lazy	Spring DI	Create bean only on demand
@DependsOn	Spring DI	Bean initialization order
@Scope	Spring DI	Bean scope (singleton/prototype/request/session)
@Value	Spring DI	Inject property value (${...})
@PropertySource	Spring Config	Load .properties

@PropertySources	Spring Config	Multiple property sources
@Import	Spring Config	Import other configs
@ImportResource	Spring Config	Import XML configs
@Profile	Spring Config	Activate bean only for profile
@ComponentScan	Spring Core	Scan packages for components
@ComponentScans	Spring Core	Multiple scans
@Order	Spring Core	Ordering filters/interceptors/beans
@Lookup	Spring DI	Method injection for prototype beans
@Required	Spring Legacy	Mandatory setter injection (older usage)
@Description	Spring Meta	Describe bean
@Role	Spring Meta	Bean role (infra/app)

---

6) Spring Boot (Auto configuration + runtime)

Annotation	**Category**	**Real-time Use Case**
@SpringBootApplication	Spring Boot	Main entry point (@Configuration + @EnableAutoConfiguration + @ComponentScan) (Home)
@EnableAutoConfiguration	Spring Boot	Enable auto-config
@SpringBootConfiguration	Spring Boot	Specialized @Configuration
@ConfigurationProperties	Boot Config	Bind properties into POJO
@EnableConfigurationProperties	Boot Config	Register config properties classes
@ConfigurationPropertiesScan	Boot Config	Scan for config properties
@AutoConfigureAfter	Boot AutoConfig	Ensure order after another auto-config

@AutoConfigureBefore	Boot AutoConfig	Ensure before another
@AutoConfigureOrder	Boot AutoConfig	Auto-config priority
@ImportAutoConfiguration	Boot AutoConfig	Import auto-config explicitly
@ConditionalOnClass	Boot Conditionals	Enable config only if class exists
@ConditionalOnMissingClass	Boot Conditionals	Enable if class absent
@ConditionalOnBean	Boot Conditionals	Enable if bean exists
@ConditionalOnMissingBean	Boot Conditionals	Enable if bean missing
@ConditionalOnProperty	Boot Conditionals	Enable based on property flag
@ConditionalOnResource	Boot Conditionals	Enable based on resource
@ConditionalOnWebApplication	Boot Conditionals	Only for web apps
@ConditionalOnNotWebApplication	Boot Conditionals	Only non-web
@ConditionalOnExpression	Boot Conditionals	SpEL condition
@ConditionalOnJava	Boot Conditionals	Java version condition
@ConditionalOnSingleCandidate	Boot Conditionals	Enable if single bean candidate
@ConditionalOnCloudPlatform	Boot Conditionals	Cloud platform-specific config
@ServletComponentScan	Boot Web	Scan servlet filters/listeners
@SpringBootTest	Boot Testing	Load full application context for tests

---

7) Spring MVC / REST APIs

Annotation	**Category**	**Real-time Use Case**
@RequestMapping	MVC	Base mapping
@GetMapping	MVC	GET endpoint
@PostMapping	MVC	POST endpoint
@PutMapping	MVC	PUT endpoint

@PatchMapping	MVC	PATCH endpoint
@DeleteMapping	MVC	DELETE endpoint
@PathVariable	MVC	Extract URI variable
@RequestParam	MVC	Query param
@RequestBody	MVC	JSON → Object
@ResponseBody	MVC	Object → JSON
@RequestHeader	MVC	Header values
@CookieValue	MVC	Cookie
@ModelAttribute	MVC	Bind form/model
@ResponseStatus	MVC	Fix status code
@CrossOrigin	MVC	CORS configuration
@ControllerAdvice	MVC	Global exception handling
@RestControllerAdvice	MVC	Advice for REST APIs
@ExceptionHandler	MVC	Handle exception in controller/advice
@InitBinder	MVC	Custom binder / validator
@SessionAttributes	MVC	Store attributes in session
@SessionAttribute	MVC	Read session attribute
@RequestAttribute	MVC	Read request attribute
@MatrixVariable	MVC	Matrix params
@ResponseBodyAdvice	MVC	Customize responses globally
@RequestPart	MVC	Multipart part binding
@ResponseEntity (not annotation)	MVC	Custom response headers/status

8) Spring AOP

Annotation	Category	Real-time Use Case
@Aspect	AOP	Mark aspect class
@Pointcut	AOP	Define pointcut expression
@Before	AOP	Run before method
@After	AOP	Run after method
@AfterReturning	AOP	Run after successful method
@AfterThrowing	AOP	Run after exception
@Around	AOP	Wrap method execution (most powerful)
@DeclareParents	AOP	Introductions

---

9) Transactions

Annotation	Category	Real-time Use Case
@Transactional	Transactions	Atomic DB operation (rollback on failure)
@EnableTransactionManagement	Transactions	Enable annotation-based TX

---

10) Spring Async / Scheduling

Annotation	Category	Real-time Use Case
@EnableAsync	Async	Enable async method execution
@Async	Async	Run method in separate thread pool
@EnableScheduling	Scheduling	Enable scheduler
@Scheduled	Scheduling	Cron/fixed-rate jobs
@Schedules	Scheduling	Multiple schedules

---

11) Spring Caching

Annotation	Category	Real-time Use Case
@EnableCaching	Cache	Enable caching
@Cacheable	Cache	Cache method result
@CachePut	Cache	Update cache
@CacheEvict	Cache	Remove cache entries
@Caching	Cache	Combine multiple cache operations
@CacheConfig	Cache	Default cache settings

---

12) Spring Data JPA (Repositories + queries)

Annotation	Category	Real-time Use Case
@EnableJpaRepositories	Data JPA	Enable repositories
@RepositoryDefinition	Data JPA	Define repository manually
@Query	Data JPA	Custom JPQL/SQL query
@Param	Data JPA	Bind query parameters
@Modifying	Data JPA	Non-select queries (update/delete)
@Transactional	Data JPA	Tx around repo/service
@EntityGraph	Data JPA	Fetch graph optimization
@Lock	Data JPA	Pessimistic/optimistic lock
@Procedure	Data JPA	Call stored procedure
@EnableJpaAuditing	Data Auditing	Auto fill created/modified fields
@CreatedDate	Auditing	Auto set created timestamp
@LastModifiedDate	Auditing	Auto set modified timestamp

@CreatedBy	Auditing	Track created user
@LastModifiedBy	Auditing	Track modified user

---

13) Spring Data MongoDB

Annotation	Category	Real-time Use Case
@Document	MongoDB	Collection mapping
@Id	MongoDB	Document id
@Field	MongoDB	Custom field name
@DBRef	MongoDB	Reference to another document
@Indexed	MongoDB	Create index
@CompoundIndex	MongoDB	Multi-field index
@CompoundIndexes	MongoDB	Many compound indexes
@TextIndexed	MongoDB	Full text index
@GeoSpatialIndexed	MongoDB	Geo index

---

14) Spring Data Redis

Annotation	Category	Real-time Use Case
@RedisHash	Redis	Store object as Redis hash
@Id	Redis	Redis key id
@Indexed	Redis	Secondary lookup
@TimeToLive	Redis	Expire entity automatically

---

15) Spring Security (Authentication / Authorization)

Annotation	Category	Real-time Use Case
@EnableWebSecurity	Security	Enable Spring Security
@EnableMethodSecurity	Security	Enable method security (new)
@EnableGlobalMethodSecurity	Security	Legacy method security
@PreAuthorize	Method Security	Check role/permission before method
@PostAuthorize	Method Security	Validate after returning
@PreFilter	Method Security	Filter input list
@PostFilter	Method Security	Filter output list
@Secured	Security	Role-based access
@RolesAllowed	JSR-250	Allow roles
@AuthenticationPrincipal	Security	Inject logged-in user
@CurrentSecurityContext	Security	Access SecurityContext in methods

---

16) OAuth2 / JWT Resource Server

Annotation	Category	Real-time Use Case
@RegisteredOAuth2AuthorizedClient	OAuth2	Get authorized client
@EnableAuthorizationServer	Legacy OAuth	Old OAuth2 authorization server
@EnableResourceServer	Legacy OAuth	Old resource server
@EnableOAuth2Client	OAuth2	Enable OAuth2 client

---

17) Microservices / Spring Cloud (Discovery, Feign, Circuit breaker, Gateway)

Annotation	Category	Real-time Use Case
@EnableDiscoveryClient	Spring Cloud	Register service into discovery server (Home)
@EnableEurekaClient	Spring Cloud Netflix	Eureka-specific registration (Cloud Spring)
@FeignClient	Feign	Declarative REST client between microservices
@EnableFeignClients	Feign	Enable Feign scanning
@LoadBalanced	Cloud	Enable client-side load balancing on RestTemplate
@RefreshScope	Config	Refresh bean config without restart
@EnableConfigServer	Config Server	Start centralized config server
@EnableCircuitBreaker	Circuit breaker	Enable circuit breaking
@EnableHystrix	Netflix Hystrix	Legacy hystrix
@EnableHystrixDashboard	Hystrix	Monitoring dashboard
@EnableZuulProxy	Zuul	Legacy gateway
@EnableGateway	Gateway	Enable API gateway
@EnableBinding	Stream	Bind channels (older stream style)
@Output	Stream	Outgoing message channel
@Input	Stream	Incoming message channel
@EnableScheduling	Cloud patterns	Scheduled jobs per service

18) Kafka (Spring Kafka) ✅

Annotation	Category	Real-time Use Case
@EnableKafka	Kafka	Enable Kafka listener detection (non-Boot sometimes) (Stack Overflow)
@KafkaListener	Kafka Consumer	Consume Kafka messages (topic/group) (Home)
@KafkaListeners	Kafka	Multiple listeners on a method/class
@TopicPartition	Kafka	Listen specific partition(s)
@PartitionOffset	Kafka	Start from offset
@KafkaHandler	Kafka	Multi-method handler inside @KafkaListener class
@SendTo	Kafka	Send consumer method return value to reply topic (Home)
@RetryableTopic	Kafka Retry	Automatic retries + DLT
@DltHandler	Kafka Retry	Handle dead-letter topic message
@EnableKafkaStreams	Kafka Streams	Enable Kafka Streams infra
@KafkaKey	Kafka	Mark key parameter in listener
@Header	Messaging	Extract Kafka header
@Payload	Messaging	Extract message payload

---

19) Spring Messaging / Integration (used in Kafka, WebSocket, events)

Annotation	Category	Real-time Use Case
@MessageMapping	Messaging	WebSocket/STOMP mapping
@SendTo	Messaging	Send to destination
@SendToUser	Messaging	Send to specific user
@Header	Messaging	Extract header

@Headers	Messaging	Extract headers map
@Payload	Messaging	Extract payload
@EventListener	Core Events	Handle Spring application events
@TransactionalEventListener	Events	Trigger only after commit/rollback

---

20) OpenAPI / Swagger (API documentation)

Annotation	**Category**	**Real-time Use Case**
@OpenAPIDefinition	OpenAPI	Global API doc config
@Info	OpenAPI	API metadata
@Tag	OpenAPI	Group endpoints
@Tags	OpenAPI	Multiple tags
@Operation	OpenAPI	Describe API operation
@Parameter	OpenAPI	Document param
@Parameters	OpenAPI	Multiple params
@RequestBody	OpenAPI	Document request body
@ApiResponse	OpenAPI	Document response
@ApiResponses	OpenAPI	Many responses
@Schema	OpenAPI	Model schema
@Hidden	OpenAPI	Hide endpoint/model
@SecurityRequirement	OpenAPI	Security requirement

---

21) Spring Test (Unit + Integration)

Annotation	Category	Real-time Use Case
@SpringBootTest	Test	Full context test
@WebMvcTest	Test	Only controller layer
@DataJpaTest	Test	Only JPA layer
@DataMongoTest	Test	Mongo slice test
@JdbcTest	Test	JDBC slice test
@JsonTest	Test	JSON serialization test
@RestClientTest	Test	RestTemplate/WebClient slice
@AutoConfigureMockMvc	Test	MockMvc injection
@MockBean	Test	Mock bean in context
@SpyBean	Test	Spy bean
@DirtiesContext	Test	Reset context between tests
@ActiveProfiles	Test	Use profile for test
@TestPropertySource	Test	Override properties
@Sql	Test	Run SQL scripts before/after test

✅ Total Covered

This master table already covers **300+ annotations** across:

- Java + Lombok
- Jakarta JPA + Validation
- Spring Core + Boot + MVC
- Spring Data (JPA/Mongo/Redis)
- Security
- Microservices (Spring Cloud)
- Kafka (Spring Kafka)
- Swagger/OpenAPI
- Testing

If you want, I can also provide:

✅ **Downloadable Excel format**
✅ **Separate tables** for *only Kafka*, *only Microservices*, *only Spring Boot*, etc.
✅ **Examples project** with code snippets showing 50+ most-used annotations in a real Spring Boot microservice